

Subject: CS336 Lecture 1, Overview and Tokenization

Date: from 2026/7/24 to 2026/7/25

Contents

Why This Course Exists	1
The Current Language-Model Landscape	2
Course Syllabus	3
basics(): A Basic Language Model	4
systems(): Squeezing the Hardware	6
scaling_laws(): Predictability over Optimality	8
data(): Capabilities and Curation	9
alignment(): Learning from Weak Supervision	10
Tokenization	11
Tokenizer 的职责	11
分词现象与压缩率	11
字符级分词器 (Character Tokenizer)	12
字节级分词器 (Byte Tokenizer)	13
词级分词器 (Word Tokenizer)	14
BPE: 原理与训练	14

Why This Course Exists

研究者与底层技术的脱节

研究界正在逐渐与支撑这项技术的底层原理脱节。从实现方式可以看到这条清晰的演化路径：

- 2016 年. 研究者自己实现并训练模型；
- 2018 年. 研究者下载模型（如 BERT）并微调；
- 今天. 研究者通过 prompt 调用 API 模型（GPT / Claude / Gemini）。

抽象层级的提升确实带来了生产力，但有两个代价：

- 抽象是泄漏的（leaky）。与编程语言或操作系统不同，这些抽象没有清晰的边界，底层细节会渗透上来；
- 仍有基础研究要做。许多根本性问题必须拆开整条技术栈（tearing up the stack）才能推进。

因此，要对这项技术形成完整理解，就必须深入底层；这也是开展基础研究的前提。

理解源于构建

本课程的核心哲学是 **understanding via building**：通过亲手实现来理解系统。

但也存在一个现实的困难：

- 前沿模型极其昂贵。训练一次 GPT-4 级别的模型耗资巨大；
- 没有公开细节。GPT-4 的技术报告几乎没有披露架构、数据与训练细节。

所以课程的策略是：在可控的规模上把核心组件与训练流程完整地实现一遍，用**构建**换取理解。

三类知识

这门课希望传授三类知识：

1. **Mechanics**（机制）。事情如何运作：什么是 Transformer，模型并行如何工作；
 2. **Mindset**（思维）。把硬件压榨到极致，严肃对待规模化；
 3. **Intuitions**（直觉）。哪些数据与建模决策能带来更好的精度。
- 可教授的部分。Mechanics 与 Mindset 可以教授，而且可以迁移；
 - 只能部分教授的部分。Intuitions 只能部分传授，且不一定能跨尺度迁移。

直觉未必可迁移

有些设计决策目前无法被严格证明，只来自实验，甚至来自运气。最著名的例子是 Noam Shazeer 提出 SwiGLU 的论文：作者在结论中坦言无法解释这些结构为何有效。

4 Conclusions

We have extended the GLU family of layers and proposed their use in Transformer. In a transfer-learning setup, the new variants seem to produce better perplexities for the de-noising objective used in pre-training, as well as better results on many downstream language-understanding tasks. These architectures are simple to implement, and have no apparent computational drawbacks. We offer no explanation as to why these architectures seem to work; we attribute their success, as all else, to divine benevolence.

Figure 1: SwiGLU 论文的结论：把成功归因于“divine benevolence”。

苦涩的教训

“The bitter lesson” 常被误读，正确的解读是：

- 错误解读. 只要 scale，算法就不重要；
- 正确解读. 能 scale 的算法才是重要的。

可以用一个简单的关系概括：

$$\text{accuracy} = \text{efficiency} \times \text{resources}$$

规模越大，效率越重要——因为在大尺度上浪费不起算力。于是问题的框架变成：给定固定的计算与数据预算，能构建出的最好模型是什么？换句话说，就是最大化效率。

The Current Language-Model Landscape

前神经时代（2010 年代之前）

- 信息论起源. 用语言模型来度量英语的熵（Shannon 1950）；
- N-gram 语言模型. 长期用于机器翻译与语音识别系统（Brants+ 2007）。

神经网络组件（2010 年代）

这一时期逐步凑齐了现代语言模型的全部零件：

- 长短期记忆网络（LSTM）；
- 第一个神经语言模型；
- 序列到序列建模（seq2seq），用于机器翻译；
- Adam 优化器；
- 注意力机制，最初用于机器翻译；
- Transformer 架构，同样最初用于机器翻译；
- 混合专家（MoE）；
- 模型并行。

早期基础模型（2010 年代末）

- **ELMo**. 用 LSTM 预训练，微调能提升下游任务；
- **BERT**. 用 Transformer 预训练，微调能提升下游任务；
- **T5 (11B)**. 把所有任务都建模成 text-to-text.

拥抱规模化

- **GPT-2 (1.5B)**. 生成流畅文本，首次出现零样本（zero-shot）能力的迹象；
- **Scaling laws**. 为规模化提供了希望与可预测性；
- **GPT-3 (175B)**. 展现出 in-context learning；
- **PaLM (540B)**. 规模巨大，但相对的训练并不充分；
- **Chinchilla (70B)**. 提出 compute-optimal 的缩放定律.

开放模型

早期复现尝试. 对 GPT-3 的早期复现包括 EleutherAI 的开放数据集（The Pile）与模型（GPT-J）、Meta 的 OPT (175B)、Hugging Face / BigScience 的 BLOOM (176B).

可信的 **open-weight** 模型（权重 + 论文）. Llama、Mistral、DeepSeek、Qwen、Kimi、GLM、MiniMax、MIMO 等. 这些模型正在逼近闭源模型（GPT / Claude / Gemini）.

完全开源（权重 + 论文 + 代码 + 数据）. AI2 的 Olmo、NVIDIA 的 Nemotron、Marin（开放开发流程）.

开放性对信任与创新都很重要；正是开放模型公开的思想，让 CS336 这样的课程得以开设.

“语言模型”的含义在变

- **2018（BERT）**. 一个用来微调的东西；
- **2020（GPT-3）**. 一个用来 prompt 的东西；
- **2022（ChatGPT）**. 一个可以对话的东西；
- **2026（agents）**. 一个可以自主行动的东西.

不变的是**基础**（attention、kernels、optimization）；变的是**规格**（更长的上下文，推理效率变得更重要）.

Course Syllabus

课程地图：五个 Assignment

全部信息见课程网站. 课程围绕五个 assignment 组织：

- **basics()**. Assignment 1: tokenization、模型架构、训练；
- **systems()**. Assignment 2: kernels、并行、推理；
- **scaling_laws()**. Assignment 3: 缩放定律；
- **data()**. Assignment 4: 评测、数据策展、变换、过滤、去重、混合；

- **alignment()**. Assignment 5: RLHF、RL 算法、RL 系统.

贯穿主线：效率

一切都围绕 **efficiency**. 资源指的是数据加上硬件（计算、内存、通信带宽）. 核心问题是：给定固定的一组资源，如何训练出最好的模型？

今天我们是算力受限的，因此所有设计决策都在回答“如何把给定硬件压榨到极致”：

- **Systems**. 显然关于效率；
- **Tokenization**. 直接处理原始字节很优雅，但对今天的模型架构而言计算效率低下；
- **Model architecture**. 许多改动都是为了减少内存或 FLOPs（例如共享 KV cache、滑动窗口注意力）；
- **Data filtering**. 避免把宝贵的算力浪费在低质量 / 无关数据上；
- **Scaling laws**. 用小模型做超参数调优，从而少花算力.

basics(): A Basic Language Model

目标与组件

目标. 能够训练一个基础的语言模型. **组件**. tokenization、模型架构、训练.

这三块分别回答了：模型操作的最小单元是什么、模型长什么样、模型的参数如何被设定.

Tokenization：模型操作的原子

定义. Tokenizer 在原始输入（字节）与整数序列（token）之间相互转换.

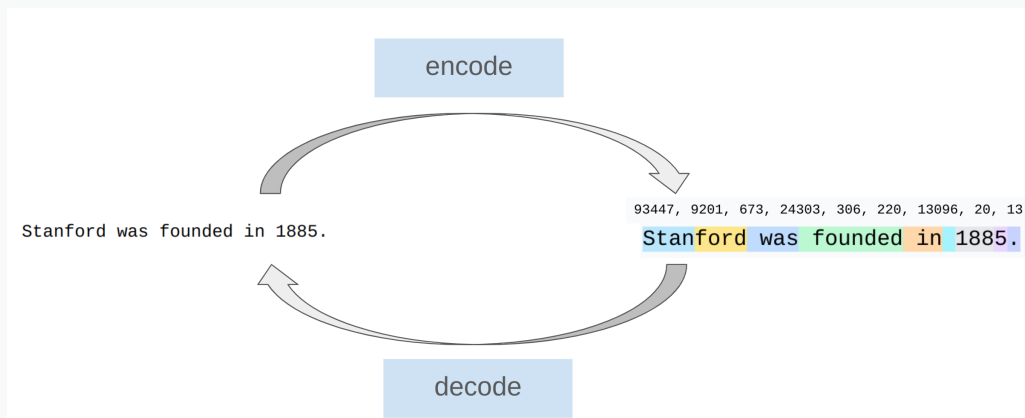


Figure 2: encode 把字符串转成 token 序列，decode 再还原.

最流行的分词算法是 **Byte-Pair Encoding (BPE)**. 直觉是把输入切成频繁出现的片段.

从效率视角看，分词带来两个好处：

- **缩短上下文长度**. 1000 字节可以压缩到约 250 个 token；
- **自适应计算**. 把更多建模容量花在输入中“有意思”的部分.

梦想. tokenizer-free 的模型架构, 直接操作字节. 这类方案很有前景, 但还没有被扩展到前沿规模.

Model architecture: 从原始 Transformer 出发

起点是原始 Transformer (Vaswani+ 2017) .

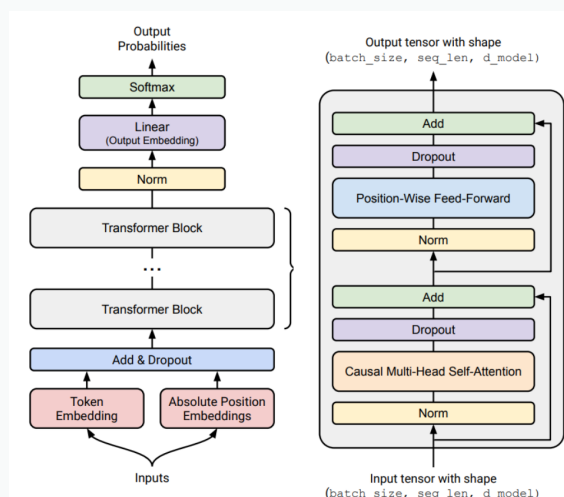


Figure 3: 原始 Transformer 的整体结构与单个 Block.

之后的改进可以按组件归类:

- 激活函数. ReLU、SwiGLU;
- 位置编码. 正弦编码、RoPE;
- 归一化. LayerNorm、RMSNorm、QK norm, 以及 pre-norm 与 post-norm 之争;
- 注意力. 全注意力、稀疏 / 局部注意力、GQA、MLA;
- 循环 / 状态空间 / 线性注意力. Mamba、Gated DeltaNet;
- MLP. dense 与 mixture of experts;
- 形状. hidden dimension、depth、head 数、expert 数.

Training: 如何设定参数

- **Loss function.** 例如 multi-token prediction;
- **Optimizer.** 例如 AdamW、SOAP、Muon;
- **Initialization scale.** 例如 Xavier init、muP;
- **Learning rate schedule.** 例如 cosine、WSD;
- **Regularization.** 例如 dropout、weight decay;
- **Batch size.** 例如 critical batch size;
- **MoE specific.** 负载均衡 (例如 aux-free) .

Assignment 1 概览

- 实现 BPE tokenizer;
- 实现 Transformer、交叉熵损失、AdamW 优化器与训练循环;
- 做资源核算 (resource accounting);
- 在 TinyStories 与 OpenWebText 上训练;
- Leaderboard: 在 B200 上给定 45 分钟, 最小化 OpenWebText 的 perplexity.

高层原则. 一切都是为了平衡下面三者:

- **Expressivity** (表达力). 能否表达数据中的复杂依赖;
- **Stability** (稳定性). 让参数与梯度范数保持在“刚刚好”的区间;
- **Efficiency** (效率). 训练与推理都能在硬件上跑得足够快.

systems(): Squeezing the Hardware

基础: 资源核算

目标. 把硬件 (GPU 或 TPU) 压榨到极致. 组件. kernels、并行、推理.

资源核算. 先刻画模型的内存与计算特征, 例如用 1T token 训练 70B 参数, 大约需要 4.2×10^{23} FLOPs:

```
total_flops = 6 * 70e9 * 1e12
```

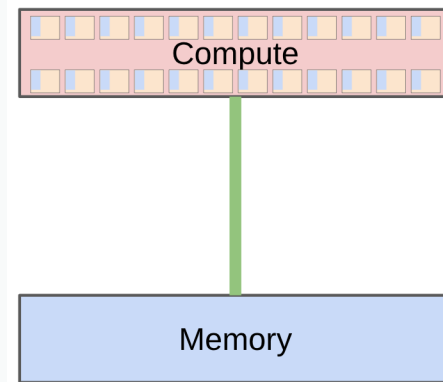


Figure 4: 计算单元与内存之间的数据搬运.

- 模型参数必须从内存 (HBM) 搬到计算单元 (SM);
- 例子: B200 在 bf16 下可达 2.25 PFLOP/s, 内存带宽约 8 TB/s;
- Roofline 分析: 判断当前是 compute-bound 还是 memory-bound;
- 基准测试与 profiling (Nsight): 看实践中真正发生了什么.

Kernels

- Kernel 是运行在 GPU 上的一个函数;
- 用 PyTorch 时, 每个基本运算都会启动一个标准 kernel;

- 可以写自定义 kernel，让 GPU 跑得更快；
- 核心原则：组织计算，最小化数据搬运。
- 朴素做法. 读 HBM；算 A；写 HBM；读 HBM；算 B；写 HBM；
- 融合做法. 读 HBM；算 A 和 B；写 HBM.

常见策略与概念：算子融合（matmul + activation）、tiling（FlashAttention）、warp divergence、memory coalescing、bank conflicts、occupancy、bulk-async memory transfers. 可以用 CUDA / Triton / CUTLASS / ThunderKittens 来写 kernel.

Parallelism

- 如果有 1024 张 GPU 呢？
- GPU 之间的数据搬运更慢，但“最小化数据搬运”的原则不变；
- 使用经典的集合通信操作（gather、reduce、all-reduce）；
- 把内存（参数、激活、梯度、优化器状态）切分到多张 GPU；
- 计算如何拆分：data / tensor / pipeline / sequence / expert parallelism.

Inference

目标. 给定 prompt 生成 token（真正把模型用起来）. 推理同样服务于强化学习、test-time compute 与评测.

推理分两个阶段：

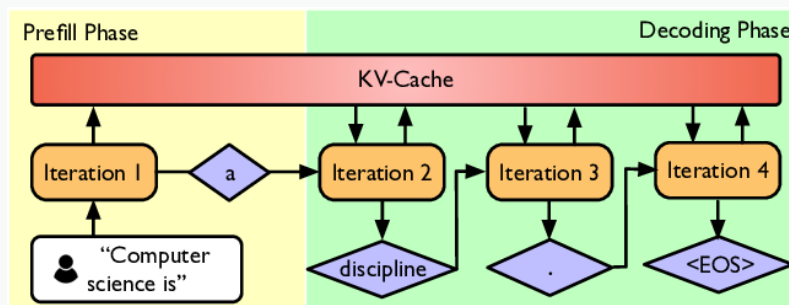


Figure 5: prefill 阶段与 decode 阶段.

- **Prefill**（类似训练）. token 已经给定，可以一次性并行处理（compute-bound）；
- **Decode**. 一次只能生成一个 token（memory-bound）.

加速 decode 的方法：

- 用更便宜的模型. 剪枝、量化、蒸馏；
- 投机解码（**Speculative decoding**）. 用一个更便宜的 draft 模型一次生成多个 token，再用完整模型并行打分（仍是精确解码）；
- 系统优化. 融合 kernel、continuous batching.

Assignment 2 与推荐读物

- 用 Triton 实现融合的 RMSNorm kernel;
- 实现分布式数据并行训练;
- 实现优化器状态分片;
- 对实现做基准测试与 profiling.

推荐读物. *How to Scale Your Model*. 它清晰地梳理了 LLM systems 的概念; 内容来自 Google, 因此以 TPU 为主, 但高层概念是相通的.

scaling_laws(): Predictability over Optimality

核心思想: scaling recipe

设定. 如果给你 10^{25} FLOPs 的算力, 你会用什么超参数来训练一个好模型?

在全尺度上做超参数调优太贵了. 关键的观念转变是: 不要只盯着单一尺度, 而要构造一个 **scaling recipe** (FLOPs $\rightarrow$ 超参数):

1. 在多个更小的尺度上做实验, 测量 loss (例如到 10^{24} FLOPs);
2. 拟合一条 scaling law, 用来预测该 recipe 在目标尺度 (例如 10^{25} FLOPs) 上的 loss.

于是可以:

- 用小尺度实验去优化面向更大尺度的 recipe;
- 在真正开跑之前就预测目标尺度的 loss.

Scaling law 不会自动出现, 它需要精心构造 recipe, 并把模型参数化得能够获得 **hyper-parameter transfer**.

可预测性与最优性至少同样重要!

算力预算下: 更大的模型, 还是更多的 token?

给定 FLOPs 预算 $C = 6ND$, 应该用更大的模型 N , 还是在更多 token D 上训练?

经典的 compute-optimal scaling laws 给出的方法:

1. 对多个较小的 FLOPs 预算, 先通过 **ISOFLP 曲线** 找到最优的 N ;
2. 再拟合 scaling law, 外推到更大的 FLOPs 预算.

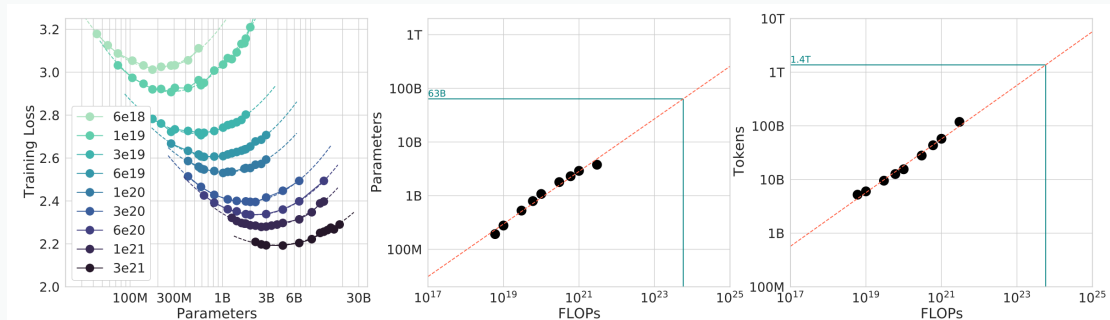


Figure 6: ISOFLP 曲线与参数 / token 的最优分配.

结论. $D \approx 20N$ 大致是最优的（例如 70B 参数的模型应在约 1.4T token 上训练）.

注意. 这一结论没有考虑推理成本——从部署角度看，往往希望模型更小.

Assignment 3 概览

- 基于历史运行定义一个训练 API（超参数 $\rightarrow$ loss）;
- 在给定 FLOPs 预算下提交“训练任务”，收集数据点;
- 用这些数据点拟合 scaling law;
- 提交外推得到的超参数与 loss 预测;
- Leaderboard: 在给定 FLOPs 预算下最小化 loss.

data(): Capabilities and Curation

评测：为什么以及怎么评

问题. 我们希望模型具备哪些能力？多语言？擅长对话？具备 agentic coding 能力？

评测的目的分两类：

1. 内部. 指导模型研发（关注跨尺度的平滑性与相对表现）;
2. 外部. 衡量真实用例中的绝对质量（关注生态效度 ecological validity）.

评测的例子：

- **Perplexity.** 最好在私有文档上评估，避免数据污染；
- 高级用例. GPQA、HLE、SWE-Bench、Terminal-Bench.

语言模型是通用系统，因此需要多样的评测集合.

数据策展

数据不会从天上掉下来. 来源包括从互联网爬取的网页、书籍、arXiv 论文、GitHub 代码等.

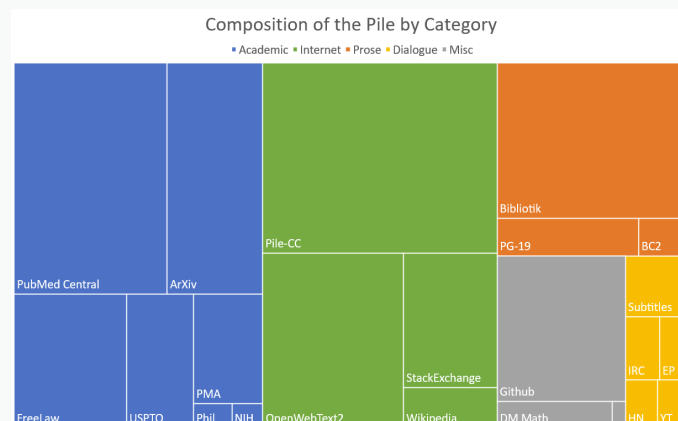


Figure 7: The Pile 各来源的构成比例.

- 用合理使用（fair use）为训练版权数据辩护？
- 也可能必须获得数据授权（例如 Google 与 Reddit 的数据协议）；
- 原始数据是 HTML、PDF、目录（而不是干净的文本），需要处理。

数据处理

- 变换（Transformation）. 把 HTML / PDF 转成文本（抽取正文）；
- 过滤（Filtering）. 用分类器保留高质量数据、去除有害内容；
- 去重（Deduplication）. 节省算力、避免记忆化；可用 Bloom filter 或 MinHash；
- 数据混合（Data mixing）. 每个来源该上采样 / 下采样多少？
- 改写 / 合成数据. 用 LM 增广真实数据，使其更接近下游任务。

按训练阶段划分的数据类型：

- 预训练数据. 规模大、多样；
- 中期训练数据. 高质量，包含长上下文；
- 后训练数据. 监督微调（对话、带工具调用的 agentic traces）。

Assignment 4 概览

- 把 Common Crawl 的 HTML 转成文本；
- 训练分类器来过滤质量与有害内容；
- 用 MinHash 做去重；
- Leaderboard: 在给定 token 预算下最小化 perplexity.

alignment(): Learning from Weak Supervision

从强监督到弱监督

到目前为止，我们都在**完整监督**下训练模型（预测下一个 token）。当模型已经足够可用之后，可以用**弱监督**进一步提升它。

为什么需要弱监督？因为评判（critique）往往比生成（generate）更容易。

基本模板：

1. 让模型生成回答；
2. 用 {人类, verifier, LM judge} 给回答打分；
3. 更新模型，使其偏向更好的回答。

算法与挑战

- PPO. 来自强化学习的 proximal policy optimization；
- DPO. 直接偏好优化，面向偏好数据，更简单；
- GRPO. 组相对偏好优化，去掉了 value function.

挑战：

- RL 算法不稳定、难调；
- 大规模训练需要大量新基础设施（异步 rollout 的推理）；
- 需要不断在系统效率与 on-policy-ness 之间权衡。

Assignment 5. 实现 DPO 与 GRPO.

Tokenization

Tokenizer 的职责

从字符串到 token

原始文本通常表示成 Unicode 字符串，例如：

```
string = "Hello, \U0001F30D! 你好!"
```

而语言模型在 token 序列上定义概率分布（token 通常用整数索引表示）：

```
indices = [15496, 11, 995, 0]
```

因此需要一个把字符串编码成 token 的过程，也需要一个把 token 解码回字符串的过程。定义一个 **Tokenizer** 就是实现了 **encode** 与 **decode** 两个方法的类。

分词现象与压缩率

几个直观观察

先感受一下 tokenizer 的行为（可玩一玩在线交互站点 [tiktokenizer](#)）：

- 词与其前面的空格属于同一个 **token**。例如 " world"；
- 词在句首与句中表示不同。例如 "hello hello"；
- 数字会被切成每几位一个 **token**。

以 OpenAI 的 GPT-5 tokenizer（tiktoken）为例：

```
tokenizer = get_gpt5_tokenizer()
string = "Hello, \U0001F30D! 你好!"

indices = tokenizer.encode(string)
reconstructed_string = tokenizer.decode(indices)
assert string == reconstructed_string
```

压缩率：每个 token 承载多少字节

压缩率定义为每个 token 对应的字节数：

```
def get_compression_ratio(string, indices):  
    num_bytes = len(bytes(string, encoding="utf-8"))  
    num_tokens = len(indices)  
    return num_bytes / num_tokens
```

压缩率越大，序列越短（这一点很关键，因为注意力的计算量随序列长度平方增长）。可以通过增大词表大小（**vocabulary size**）来提高压缩率，但会带来稀疏性。

词表大小与稀疏性

词表大小指 tokenizer 所有可能出现的 token 的总数量。

- 词表很小（例如只有 26 个字母加空格）时，任何英文文本都会被切成单个字母，token 数量非常多；
- 词表很大（例如 10 万个常用词、词根、标点）时，常见的词可以整体作为一个 token，token 数量少很多。

增大词表 → 提高压缩率。 如果“人工智能”被整个收录为一个 token，它只占 1 个 token；若词表小，则可能被切成“人工”“智能”两个 token。词表越大，常见的词与短语越可能被整体编码，序列越短。

压缩率提高 → 稀疏性。 当词表非常大（例如从 3 万扩到 10 万），每个 token 的“身份”都是一个超高维的 one-hot 向量：只有当前 token 对应的那一维是 1，其余几万维都是 0。这带来两个后果：

- 输入层变得高度稀疏；
- 输出层（Softmax）计算量暴涨，而且概率高度集中在少数 token 上，其余大量 token 概率接近 0，分布也是稀疏的。

字符级分词器（Character Tokenizer）

用 Unicode code point 表示字符

Unicode 字符串是 Unicode 字符的序列。每个字符可以通过 ord 转成整数 code point：

```
assert ord("a") == 97  
assert ord("\U0001F30D") == 127757
```

再通过 chr 转回来：

```
assert chr(97) == "a"  
assert chr(127757) == "\U0001F30D"
```

基于此可以构建一个 CharacterTokenizer 并验证往返：

```
tokenizer = CharacterTokenizer()
string = "Hello, \U0001F30D! 你好!"
indices = tokenizer.encode(string)
reconstructed_string = tokenizer.decode(indices)
assert string == reconstructed_string
```

两个致命问题

Unicode 字符大约有 15 万个，因此词表的下界为：

```
vocabulary_size = max(indices) + 1
```

- 问题 1：词表太大。直接浪费输出层的计算资源；
- 问题 2：许多字符非常罕见。例如 🌐，这是对词表的低效利用。

词表大却不缩短序列。“增大词表提高压缩率”的前提是词表里收录了常见的词或子词，可以把多个字符打包成一个 token。字符级分词器完全不同：

- 它的词表里没有“词”，只有单个字符（a、b、你、🌐 等）；
- 无论词表是 15 万还是 150 万，“Hello”都必须切成 H e l l o 共 5 个 token，因为基本单位就是字符；
- 它无法像 BPE 那样把“Hello”整体作为一个 token。因此词表再大也没有增加“打包”能力，每个 token 的信息密度极低——巨大的词表只是浪费了输出层算力，却没有缩短序列。

罕见字符浪费参数。为了编码 🌐，词表必须为它预留一个 ID；但它在训练语料中可能只出现几次，对应的 embedding 几乎得不到充分训练，浪费了模型参数。相比之下，子词分词器遇到罕见字符会把它拆成更基础的字节（如 UTF-8 字节），用已有的字节 token 组合表示，无需为罕见字符专门增加词表，更加灵活高效。

综合起来，字符级分词器是“两头不讨好”：词表大，压缩率低。

字节级分词器（Byte Tokenizer）

优势与致命缺陷

优势：

- 词表极小。只有 256 个 token（对应 0-255 的所有字节值）；
- 覆盖所有文本。任何 Unicode 字符串都可以用 UTF-8 编码成字节序列，因此永远不会遇到未知字符（UNK）。

致命缺陷：

- 压缩率恒为 1。每个 token 永远只代表 1 个字节；
- 对于“Hello, 🌐! 你好!”这串文本：
 - “Hello,!” 这些 ASCII 字符，每个占 1 字节 → 1 个 token；
 - “🌐”用 UTF-8 编码占 4 字节 → 需要 4 个 token；
 - 中文字符“你好”，每个在 UTF-8 中占 3 字节 → 各需要 3 个 token。

结果. 一段文本编码后的 token 序列长度, 约等于其 UTF-8 字节长度. 对于包含大量非 ASCII 字符 (中文、Emoji 等) 的文本, token 数量会远超字符数.

词级分词器 (Word Tokenizer)

按词切分

一种更接近经典 NLP 的做法是把字符串切成单词:

```
string = "I'll say supercalifragilisticexpialidocious!"  
chunks = regex.findall(r"\w+|.", string)
```

这个正则把所有字母数字字符连在一起 (即单词). 要把它变成 Tokenizer, 需要把每个 chunk 映射成整数.

优点. 每个 token 都有意义 (因为单词是人发明的).

缺点:

- 压缩率不错, 但词表可能非常巨大;
- 许多词很罕见, 模型学不到多少东西;
- 词表大小不固定;
- 训练时没见过的词只能映射到特殊的 UNK token, 既不优雅, 也会破坏 perplexity 的计算.

BPE: 原理与训练

Byte Pair Encoding 的核心思想

BPE 最早由 Philip Gage 在 1994 年提出, 用于数据压缩; 后来被引入 NLP 用于神经机器翻译 (此前论文多用基于词的分词), 再后来被 GPT-2 采用.

基本思想. 在原始文本上训练 tokenizer, 构造一个贴合数据的词表. 直觉是: 常见的字节序列用一个 token 表示, 罕见的序列用多个 token 表示.

算法草图. 从每个字节作为一个 token 开始, 不断合并当前最频繁的相邻 token 对.

训练与使用

训练:

```
string = "the cat in the hat"  
params = train_bpe(string, num_merges=3)
```

使用训练好的参数对新文本编码:

```
tokenizer = BPETokenizer(params)
string = "the quick brown fox"
indices = tokenizer.encode(string)
reconstructed_string = tokenizer.decode(indices)
assert string == reconstructed_string
```

训练算法

```
def train_bpe(string, num_merges):
    indices = list(map(int, string.encode("utf-8")))
    merges = {}
    vocab = {x: bytes([x]) for x in range(256)}
    for i in range(num_merges):
        counts = count_adjacent_pairs(indices)
        pair = max(counts, key=counts.get)
        new_index = 256 + i
        merges[pair] = new_index
        vocab[new_index] = vocab[pair[0]] + vocab[pair[1]]
        indices = merge(indices, pair, new_index)
    return BPETokenizerParams(vocab=vocab, merges=merges)

def count_adjacent_pairs(indices):
    counts = defaultdict(int)
    for index1, index2 in zip(indices, indices[1:]):
        counts[(index1, index2)] += 1
    return counts
```

把它读成下面几步：

1. 把字符串编码成字节索引列表，并从 256 个单字节 token 开始；
2. 统计所有相邻 token 对的出现次数，选出最频繁的一对；
3. 为它创建一个新 token，记录合并规则，并更新词表；
4. 在序列上执行这次合并，重复 num_merges 次。

Assignment 1 的扩展方向

上面的代码只是骨架，Assignment 1 会在此基础上要求：

- encode() 目前会遍历所有 merge，应只遍历真正相关的 merge；
- 检测并保留 special token（例如 <|endoftext|>）；
- 使用 pre-tokenization（例如 GPT-2 tokenizer 的正则）；
- 尽量把实现做得更快。